Bittenbin: A decentralized intelligence system for probabilistic events

Calvin Lin
cl@bittenbin.com

Oct 2026

Abstract

A useful intelligence system would require many independent forecasters whose answers are costly to give and can be combined into one probability. We propose a system like this on Ethereum, where a wallet creates a probabilistic neuron by solving a proof-of-work for a hash that begins with a Pythagorean triple. When a customer asks a question, each wallet stores it under the neuron that is closest to what it believes, and the protocol combines these neurons into an aggregated probability weighted by their work. Importantly, a neuron has a finite memory, so to maximize revenue, it only stores a question when it expects to be close to the final answer. Together, the neurons morph into a decentralized probabilistic brain governed by elegant and rigorous Euclidean geometry instead of centralized platforms.

1 Introduction

Probabilistic forecasts are produced everywhere today by analysts, markets and increasingly AI models. These work well for routine decisions, but a more consequential one needs a probability that is costly to give and does not rest on a single source. Prediction markets offer this by requiring skin in the game, but only for questions that can be settled. A centralized platform could instead combine many forecasters on any question, at the cost of becoming a gatekeeper that decides whose forecast counts.

What we need instead is an intelligence system where a probability is costly to give and combined across many independent forecasters without a centralized platform. We propose such a system based on a proof-of-work [1] with a geometric twist. Concretely, a wallet creates a neuron by searching for a hash that begins with a Pythagorean triple, and the direction of this triple on the Cartesian plane is the probability that the neuron stands for. When a customer asks a question, each wallet stores it under the neuron that is closest to what it believes, and the neurons that store it are added together into one answer, weighted by their work. The result is a network that answers any question with a probability, built bottom-up by those who are willing to prove their work. It can be thought of as a hash-based counterpart to models that return a probability for a query (e.g., Jev), for decisions that are too consequential to leave to a single source.

2 Neurons

We begin by defining a neuron as a unique location on the Cartesian plane. To create a neuron, a wallet first chooses a Pythagorean lattice point (x, y, C), where:

$$x^2 + y^2 = C^2, \;\; x, y, C \in \mathbb{N}$$

The location gives the probability that the neuron stands for on any Yes/No question. Since $(x/C)^2 + (y/C)^2 = 1$, we have:

$$P(\text{Yes}) = \left(\frac{x}{C}\right)^2, \quad P(\text{No}) = \left(\frac{y}{C}\right)^2$$

The wallet then searches for a nonce such that the hash

$$H = \text{keccak256}(\text{wallet}, x, y, C, \text{nonce})$$

begins with x, then y, then C, each written in binary one after another.

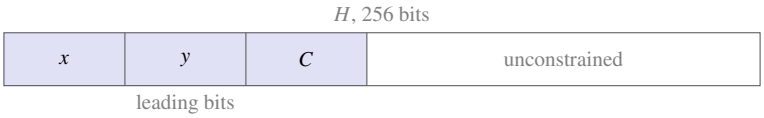


Figure 1. *A hash creates a neuron if it begins with the Pythagorean triple* (x, y, C) *that the wallet has chosen.*

For example, suppose Alice wants a neuron that stands for 36%. She chooses the location $(15, 20, 25)$, since $(15/25)^2 = 36\%$. In binary, $x = 1111$, $y = 10100$ and $C = 11001$, so she needs a hash that begins with the 14 bits 1111 10100 11001. It needs to be noted that a neuron stays at its location forever, and each lattice point can only hold one neuron. The point $(15, 20)$ cannot be used to create another neuron after Alice's proof-of-work. A wallet can create as many neurons as it wishes, and a wallet that wants to express many different beliefs needs neurons in many different directions.

3 Difficulty and Rewards

The network's difficulty comes from the lattice point itself. Every bit of x, y and C must appear at the start of the hash, so a point whose three numbers take ℓ bits in total needs 2^ℓ hashes on average.

Location	**Digits**	**Bits** ℓ	**Hashes**
$(3, 4, 5)$	1	8	256
$(15, 20, 25)$	2	14	16,384
$(150, 200, 250)$	3	24	2×10^7
$(1500, 2000, 2500)$	4	34	2×10^{10}
$(15000, 20000, 25000)$	5	44	2×10^{13}
$(150000, 200000, 250000)$	6	54	2×10^{16}
$(1500000, 2000000, 2500000)$	7	64	2×10^{19}

Table 1. *Expected work to create a neuron at locations of one to seven digits. All seven are multiples of* $(3, 4, 5)$ *and stand for* 36%.

The cheapest neurons are those at the smallest Pythagorean lattice points, and each point can only be used once. As these are taken, a new neuron must use a larger point, which takes more work. The work needed to create a neuron therefore rises with the number of neurons that already exist, such that difficulty follows adoption.

Hypotenuse below	**Lattice points**	**Longest proof**
2^8	338	24 bits
2^{12}	9,060	36 bits
2^{16}	202,728	48 bits
2^{20}	4,169,516	60 bits

Table 2. *Lattice points available as the hypotenuse grows.*

Creating a neuron rewards the wallet with the protocol's ERC-20 token Tenbinium (**TBN**), which is fair-launched with an initial supply of zero. The emission for a neuron depends only on how many neurons the protocol has accepted before it. Since the difficulty of a proof already rises with adoption, the reward does not need to fall steeply to make TBN scarcer over time. It is 2 TBN for each of the first 9,000,000 neurons, which emits 18,000,000 TBN, and 1 TBN for the million after that. From 10,000,000 neurons onwards, it halves with every 2,000,000 neurons that follow, so that the total supply approaches 21,000,000 units.

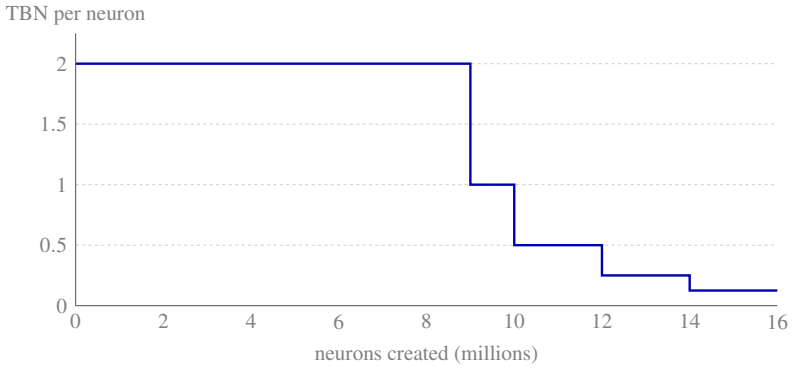


Figure 2. *TBN emission schedule. The reward per neuron holds at 2 TBN for the first nine million neurons, and then halves.*

4 Questions

The core use of TBN is to ask questions. A customer asks the network a **question** Q, whose answer is either "Yes" or "No". Any wallet that owns neurons can answer with its perceived probability on the question. It does so by **storing** the question under a neuron that it owns, usually the one whose probability is closest to what the wallet believes. For example, a neuron at $(15, 20, 25)$ stores a question when its wallet thinks that the answer is close to 36%.

Steps	**Description**
1. Ask	A customer submits a question together with a deadline, and pays the fee in TBN that the deadline sets
2. Store	Before the deadline, each participating wallet stores the question under a neuron that it owns in a single public transaction, together with any opinion that it wishes to attach
3. Aggregate	After the deadline, the neurons that stored the question give the final probability, and each of their wallets claims its share of the fee

Table 3. *Steps involved in answering a question.*

The customer chooses how long a question stays open, and this sets what it costs. There are four durations to choose from, each with a fixed fee in TBN.

Duration	**Fee**
1 hour	1 TBN
4 hours	2 TBN
1 day	5 TBN
7 days	13 TBN

Table 4. *The fee of a question by how long it stays open.*

A harder question deserves more time, and more time lets more neurons respond, so intuitively a longer question costs more. The fee grows approximately with the square root of the duration, much as the value of a financial option grows with its time to expiry. It does not change as the network grows, since a neuron's reward and its memory come from the same work. It cannot be raised in any other way, so a customer who wants more neurons to respond asks for more time.

Storing is public and takes a single transaction. The answer forms in the open as neurons store the question, much as a price forms in a market, and a wallet can attach an **opinion** to what it stores, allowing customers to receive not only the aggregated probability but also the reasoning behind it. A question that no neuron stores returns its fee to the customer.

Questions are never settled. The answer of the network is the considered opinion of the neurons that were willing to commit the question to memory, and the protocol does not measure whether it was right. Over time, the plane becomes a public memory of everything that the network has been asked, arranged by how likely each question was judged to be.

5 Memory

Importantly, a neuron has a finite memory. It can store one question for every bit of its proof, such that a neuron at (x,y,C) whose three numbers take ℓ bits in total can store at most ℓ questions. Alice's neuron at $(15,20,25)$ took a proof of 14 bits and can therefore store 14 questions. Each additional bit doubles the work behind a neuron and adds one question to its memory. A wallet is free to store a question under more than one of its neurons, which lets it state a probability that lies between them (e.g., storing a question under the neurons at $(15,20,25)$ and $(20,15,25)$, which stand for 36% and 64%, results in a probability of 50%).

A finite memory is what gives a neuron a reason to store carefully. If memory were unlimited, the best response of any neuron would be to store every question, since any share of any fee is better than none. Storing a question costs nothing in TBN, but with a finite memory, a slot that is given to one question cannot be given to another, so a wallet only stores a question whose fee is worth that slot. This keeps the network in balance without anyone managing it, such that a well-funded question attracts many neurons while a trivial one attracts few.

6 Aggregated Probability

Each neuron that stores a question is a vector (x_i, y_i) on the plane. The protocol adds these vectors together, and reads the answer from the direction of their sum, namely:

$$\mathbf{q} = \left(\sum_i x_i, \sum_i y_i \right), \qquad P(\text{Yes}) = \left(\frac{q_x}{||\mathbf{q}||} \right)^2$$

When several neurons store a question, a neuron with a larger hypotenuse has a longer vector and moves the sum further, so the answer is weighted by the hypotenuse of each neuron, and through it by the work that was needed to create it. This is similar to how a population of neurons is understood to represent a quantity, where each

neuron has a preferred value and the population is read by adding up the neurons that respond [2].

Importantly, influence over an answer is costly to gain. The work behind a neuron grows with the cube of its hypotenuse, while its weight in the sum grows only with the hypotenuse itself, so doubling the weight of a neuron takes about eight times the work. A wallet cannot avoid this by creating many small neurons, since each lattice point can only be used once and the smallest are taken first.

Stating a probability precisely takes work, and a probability near 0% or 100% can only be stated precisely, so that a more confident "Yes" or "No" is inherently more costly to state. Concretely, the smallest neurons stand for probabilities with less precision, such as 36% at $(3,4,5)$ and 92.2% at $(24,7,25)$, while a neuron that stands for about 0.1% needs a proof of 31 bits, such as $(65,2112,2113)$. The sum of vectors always lies between its parts, so a wallet cannot reach an extreme by storing a question under several smaller neurons.

For example, suppose a customer asks *"Will Bitcoin be at or above $100,000 by 31 December 2026?"*, and that three wallets answer. Alice stores the question under her neuron at 36%, Carol under a neuron at 92%, and Bob under a much larger neuron near 50%.

Wallet	**Neuron**	P	**q after**	P(**Yes**) **after**
Alice	$(15,20,25)$	36.0%	$(15,20)$	36.0%
Carol	$(24,7,25)$	92.2%	$(39,27)$	67.6%
Bob	$(119,120,169)$	49.6%	$(158,147)$	53.6%

Table 5. *Three neurons storing the same question and the aggregated probability after each.*

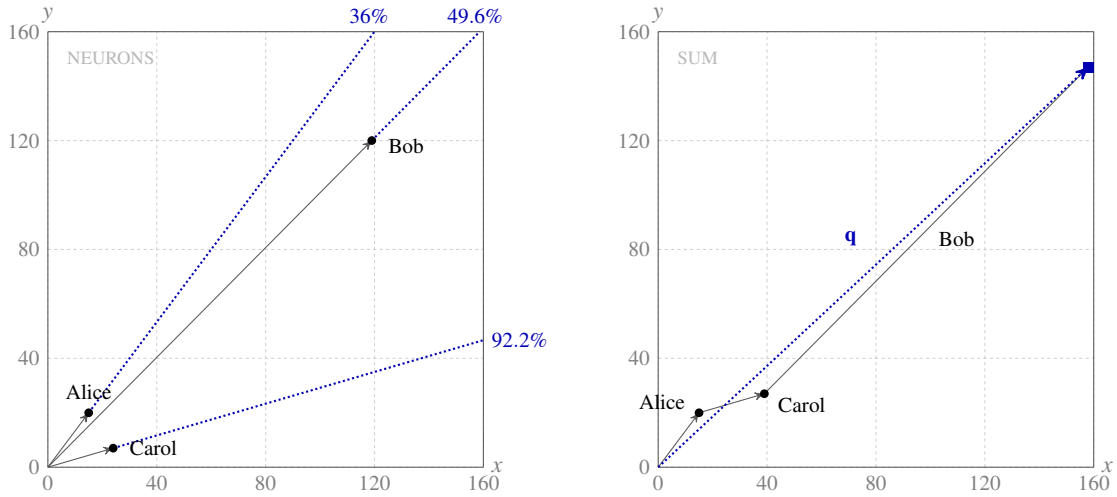


Figure 3. *In the left panel, the three neurons that stored the question are shown as vectors from the origin, each with a dotted line along the probability that it stands for. In the right panel, the same vectors are added head to tail, and the direction of their sum* $\mathbf{q}$ *gives the aggregated probability of* 53.6%.

The fee of a question is shared among the neurons that stored it, according to how much each of them contributed to the length of $\mathbf{q}$. The contribution of a neuron is its own vector measured along the direction of the sum:

$$c_i = \frac{x_i\,q_x + y_i\,q_y}{||\mathbf{q}||}, \qquad \sum_i c_i = ||\mathbf{q}||$$

so that neuron i receives $c_i/||\mathbf{q}||$ of the fee. A neuron that points the same way as the network contributes its full hypotenuse, while one that points away from it contributes less. This is what the Spherical market scoring rule [3] [4] [5] [6] would pay if every neuron had bought its own vector and the market were settled at the network's own answer.

Wallet	Neuron	Hypotenuse	Contribution	Share of fee
Alice	$(15, 20, 25)$	25	24.6	11.4%
Carol	$(24, 7, 25)$	25	22.3	10.4%
Bob	$(119, 120, 169)$	169	168.9	78.2%

Table 6. *What each neuron receives from the fee. The contributions add up to* $||\mathbf{q}|| = 215.8$.

Alice and Carol have the same hypotenuse, but Carol's neuron is further from the aggregated answer, so she receives a slightly smaller share. The difference is mild by design, so that a neuron that disagrees a little loses almost nothing and there is room for honest dissent.

7 Conclusion

We have proposed a decentralized intelligence system in which many independent forecasters answer a question as one. The protocol begins with a proof-of-work whose solutions begin with Pythagorean triples, a simple and rigorous way to give every neuron a fixed probability and a cost that rises with adoption. A customer pays in the protocol's token TBN to ask a question, and the neurons that store it are added together into a single probability, weighted by their work. Each neuron has only a finite memory, so it stores a question only when it expects to be close to the final answer, with no trusted third parties. Our hope is that, as neurons accumulate, this network can one day become a common source of knowledge for the consequential decisions of both humans and agents.

Acknowledgements

Special thanks to my friend Edward Chen (X handle *@rtedwardchen*) for his feedback and review. I am indebted to him for his loyal assistance and valuable suggestions.

We are also grateful to Satoshi Nakamoto, Vitalik Buterin and many other pioneers of decentralized finance for their inspiration.

Disclaimer

This paper is for general information purposes only. It is not investment, financial, legal, accounting or tax advice, nor is it a recommendation or solicitation to buy, sell or hold any token or other asset. Readers should not rely on it when making investment decisions.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] A. P. Georgopoulos, A. B. Schwartz, and R. E. Kettner, "Neuronal population coding of movement direction," *Science*, vol. 233, no. 4771, pp. 1416–1419, 1986.

[3] T. B. Roby, "Belief states and the uses of evidence," *Behavioral Science*, vol. 10, no. 3, pp. 255–270, 1965.

[4] I. J. Good, "Comments on 'measuring information and uncertainty' (by R. J. Buehler)," *Foundations of Statistical Inference*, pp. 337–339, 1971.

[5] R. Hanson, "Logarithmic market scoring rules for modular combinatorial information aggregation," *Journal of Prediction Markets*, vol. 1, no. 1, pp. 3–15, 2003.

[6] Y. Chen and D. M. Pennock, "A utility framework for bounded-loss market makers," *Proceedings of the 23rd Conference on Uncertainty in Artificial Intelligence*, pp. 49–56, 2007.